



Test Driven Development en Java

Jour 1 — Fondamentaux & Outillage

Formation professionnelle · 3 jours · Niveau intermédiaire



Programme — Jour 1

01 Introduction & contexte

02 Le test dans le développement

03 Origine et philosophie du TDD

04 Le cycle Red-Green-Refactor

05 Les 3A — Arrange, Act, Assert

06 Gestion des exceptions

07 Refactoring et conception émergente

08 Framework JUnit 5

09 TestNG et alternatives

10 Bonnes pratiques JUnit

11 TDD & pratiques Agiles

12 Travaux pratiques

| 01 · Introduction & Contexte

Pourquoi tester ? Pourquoi TDD ?

Objectifs de la formation

- **Maîtriser** la démarche TDD et sa mise en œuvre concrète
- **Intégrer** les tests dans le cycle de développement Java
- **Prendre en main** les outils de tests et d'intégration continue
- **Appliquer** les bonnes pratiques Agiles liées aux tests
- **Écrire** des tests de qualité avec JUnit, Mockito et FitNesse

"Le code non testé est du code bogué en attente." — Jeff Atwood

Public & Prérequis

Public concerné

- Ingénieurs en développement logiciel
- Chefs de projets techniques

Prérequis

- Connaissance de la programmation Orientée Objet
- Maîtrise des bases de Java (classes, interfaces, héritage)
- Notions de Maven ou Gradle recommandées

Programme des 3 jours

Jour 1

Définition & principes du TDD, JUnit 5, bonnes pratiques Agiles

Jour 2

Objets Mock & Stub, Mockito, techniques d'écriture de tests

Jour 3

Code hérité, FitNesse, outils d'intégration continue, couverture

Format

50 % théorie · 50 % travaux pratiques intensifs sur projets Java réels

| 02 · Le Test dans le Développement

Processus, qualité et typologies

Pourquoi tester ?

- **Détecter les bugs tôt** : corriger un bug coûte 10× moins en phase dev qu'en production
- **Documenter le comportement** : les tests décrivent ce que le code *doit* faire
- **Faciliter le refactoring** : changer le code sans crainte de régression
- **Améliorer la conception** : le code testable est souvent mieux conçu
- **Réduire la dette technique** : éviter l'accumulation de code fragile

Le coût des bugs selon la phase

Phase de détection	Coût relatif
Spécification	×1
Développement	×5
Tests d'intégration	×10
Tests système	×15
Recette client	×25
Production	×50 à ×200

Investir dans les tests est un choix économique rationnel.

Le Processus de Développement

Cycle classique (Waterfall)

- Spécifications → Conception → Codage → Tests → Livraison
- Les tests sont *après* le code → découverte tardive des bugs

Cycle Agile avec TDD

- Tests *avant* le code → feedback immédiat
- Itérations courtes → intégration continue
- Qualité intégrée dès le départ

Qualité externe

(visible par l'utilisateur)

- Fiabilité (absence de bugs)
- Performance et réactivité
- Ergonomie et utilisabilité
- Disponibilité et résilience

Qualité interne

(visible par le développeur)

- Maintenabilité du code
- Lisibilité et compréhension
- Testabilité de l'architecture
- Extensibilité et flexibilité

Pyramide des Tests



- **Tests unitaires** : méthodes et classes isolées
- **Tests d'intégration** : interactions entre composants
- **Tests E2E** : flux complets de bout en bout

Typologie des Tests (1/2)

Tests fonctionnels

- **Unitaires** : valident une unité de code (méthode/classe)
- **Intégration** : valident l'interaction entre modules
- **Système** : valident le comportement global de l'application
- **Acceptation (UAT)** : validés par le client ou le métier

Typologie des Tests (2/2)

Tests non-fonctionnels

- **Performance** : temps de réponse, débit, charge
- **Sécurité** : vulnérabilités, injections, authentification
- **Accessibilité** : conformité aux normes (WCAG)
- **Compatibilité** : différents environnements, navigateurs

Tests de régression

- Ré-exécutés à chaque modification pour détecter les régressions

Les Tests Unitaires en détail

Caractéristiques FIRST

- **F**ast — s'exécutent en millisecondes
- **I**ndependent — indépendants les uns des autres
- **R**epeatable — même résultat à chaque exécution
- **S**elf-Validating — résultat pass/fail automatique
- **T**imely — écrits *avant* ou juste *après* le code

✓ Ce qu'est un bon test unitaire

- **Teste une seule chose** : une méthode, un comportement précis
- **Lisible** : son nom décrit clairement le scénario et le résultat
- **Rapide** : pas d'I/O, pas de réseau, pas de base de données
- **Isolé** : utilise des mocks pour les dépendances externes
- **Déterministe** : pas aléatoire, pas dépendant de l'heure

03 · Origine et Philosophie du TDD

L'agilité et les tests

Histoire du TDD

- **1999** : Kent Beck formalise le TDD dans *Extreme Programming Explained*
- **2002** : Publication de *Test-Driven Development: By Example* (Kent Beck)
- **2003** : Adoption massive dans la communauté Agile
- **2004** : JUnit 4 popularise les tests en Java
- **2006** : BDD (Behavior Driven Development) émerge comme extension
- **2016** : JUnit 5 apporte une architecture modulaire moderne

"Write tests. Not too many. Mostly unit." — Guillermo Rauch

Les manifeste Agile (2001) valorise

- Les individus et leurs interactions
- Des logiciels opérationnels
- La collaboration avec le client
- L'adaptation au changement

TDD incarne ces valeurs

- Feedback rapide par les tests automatisés
- Code fonctionnel prouvé à tout moment
- Tests comme spécifications partagées
- Refactoring sûr grâce au filet de sécurité

Extreme Programming & TDD

Les pratiques XP liées aux tests

- **TDD** : test avant le code
- **Pair Programming** : revue continue du code
- **Continuous Integration** : tests automatisés à chaque commit
- **Refactoring** : amélioration continue du design
- **Simple Design** : le code le plus simple qui passe les tests

VS TDD vs Tests traditionnels

● Approche classique

- On code d'abord
- On teste ensuite
- Difficulté à tester du code non-conçu pour ça
- Tests souvent incomplets
- Refactoring risqué

● Approche TDD

- On teste d'abord
- Le test guide la conception
- Code naturellement testable
- Couverture complète par construction
- Refactoring sécurisé

Le TDD comme outil de conception

"TDD is not about testing. It's about design." — Michael Feathers

- Le test force à **réfléchir à l'API** avant l'implémentation
- Un code difficile à tester révèle un **problème de conception**
- Les interfaces émergent naturellement des besoins de test
- Le **couplage fort** devient visible immédiatement

Les Bénéfices du TDD

Bénéfices immédiats

- Détection précoce des bugs
- Documentation vivante et exécutable
- Confiance lors des modifications

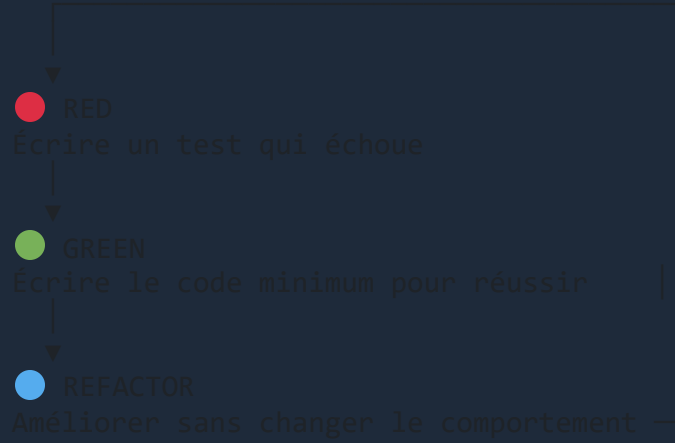
Bénéfices à long terme

- Meilleure architecture (faible couplage)
- Réduction de la dette technique
- Onboarding facilité pour les nouveaux développeurs

| 04 · Le Cycle Red-Green-Refactor

Le cœur du TDD

Le Cycle TDD



● Phase RED — Écrire le test

Règles de la phase RED

- Écrire **un seul test** à la fois
- Le test doit **échouer** (sinon il ne teste rien de nouveau)
- Le test doit **compiler** (erreur de compilation = test rouge aussi)
- Décrire le **comportement attendu** de façon précise

Exemple

```
@Test
void calculerTVA_TauxNormal_RetourneValeurCorrecte() {
    CalculateurTVA calc = new CalculateurTVA();
    double resultat = calc.calculer(100.0, 0.20);
    assertEquals(20.0, resultat, 0.001);
}
```

● Phase GREEN — Faire passer le test

Règles de la phase GREEN

- Écrire le **minimum de code** pour faire passer le test
- Ne pas chercher à écrire du code parfait
- La solution peut être "moche" ou naïve
- L'objectif : **voir le test passer au vert**

Exemple (implémentation minimale)

```
public class CalculateurTVA {  
    public double calculer(double montant, double taux) {  
        return montant * taux;  
    }  
}
```

● Phase REFACTOR — Améliorer le code

Règles de la phase REFACTOR

- Améliorer la **lisibilité** et la **structure** du code
- Ne **pas changer le comportement** (les tests doivent toujours passer)
- Éliminer la **duplication** (principe DRY)
- Améliorer les **noms** des variables, méthodes, classes

Exemple (après refactoring)

```
public class CalculateurTVA {  
    public double calculer(double montantHT, double tauxTVA) {  
        validerTaux(tauxTVA);  
        return montantHT * tauxTVA;  
    }  
    private void validerTaux(double taux) {  
        if (taux < 0 || taux > 1) throw new IllegalArgumentException("Taux invalide");  
    }  
}
```

⚡ Cadence du cycle TDD

Cycles ultra-courts

- Chaque cycle dure **2 à 10 minutes**
- Feedback **immédiat** et **constant**
- Jamais plus de quelques minutes sans voir du vert

Rythme recommandé

1. RED → 2-3 min (écrire le test)
2. GREEN → 1-3 min (implémentation minimale)
3. REFACTOR → 2-5 min (amélioration)
4. Relancer **tous** les tests avant de passer au suivant

La Règle des Petits Pas (Baby Steps)

"One test at a time, one assert at a time."

- Décomposer chaque fonctionnalité en **petits incréments**
- Chaque pas est **vérifiable** et **réversible**
- En cas d'échec, le problème est **localisé** facilement
- Accumulation de petites victoires → grande fonctionnalité

| 05 · Les 3A — Arrange, Act, Assert

Structure d'un test unitaire

Le Patron AAA

Arrange — Préparer le contexte

Initialiser les objets, les données, les mocks nécessaires au test

Act — Exécuter l'action

Appeler la méthode ou le comportement à tester (une seule chose)

Assert — Vérifier le résultat

Vérifier que le résultat correspond aux attentes



Exemple complet — AAA

```
@Test
void deposer_MontantPositif_AugmenteLesolde() {
    // ARRANGE
    CompteBancaire compte = new CompteBancaire();
    compte.setSolde(500.0);
    double montantDepose = 200.0;

    // ACT
    compte.deposer(montantDepose);

    // ASSERT
    assertEquals(700.0, compte.getSolde(), 0.001);
}
```

Exemple Given-When-Then (BDD style)

```
@Test
void givenCompteAvec500Euros_whenDeposer200_thenSoldeEst700() {
    // Given – un compte avec 500€
    CompteBancaire compte = new CompteBancaire(500.0);

    // When – on dépose 200€
    compte.deposer(200.0);

    // Then – le solde est de 700€
    assertEquals(700.0, compte.getSolde(), 0.001);
}
```

Le style Given/When/Then est équivalent à AAA et favorise la lisibilité BDD

Nommage des tests

Convention recommandée

```
methode_Contexte_ResultatAttendu()
```

Exemples de bons noms

```
calculerTVA_MontantNegatif_LeveException()  
connexion_MotDePasseIncorrect_RetourneFalse()  
retirer_SoldeSuffisant_DebiteLeCompte()  
retirer_SoldeInsuffisant_LeveInsufficientFundsException()
```

✗ Mauvais nommage à éviter

Noms vagues ou peu informatifs

```
// ✗ Mauvais
```

```
test1()
```

```
testCalculer()
```

```
testOk()
```

```
monTest()
```

```
// ✓ Bon
```

```
calculerTVA_TauxZero_RetourneZero()
```

```
calculerTVA_TauxNormal_RetourneValeurCorrecte()
```

Une assertion par test

Principe

- Chaque test vérifie **une seule chose**
- En cas d'échec, le problème est **immédiatement identifiable**

Contre-exemple

```
// ✗ Trop d'assertions dans un seul test
@Test
void testCompteBancaire() {
    CompteBancaire c = new CompteBancaire();
    assertEquals(0, c.getSolde());
    c.deposer(100);
    assertEquals(100, c.getSolde());
    c.retirer(50);
    assertEquals(50, c.getSolde());
    // Si la 2e assertion échoue, les 3e et 4e ne sont pas exécutées
}
```

Indépendance des tests

Principe

- Chaque test doit pouvoir s'exécuter **dans n'importe quel ordre**
- Pas d'état partagé entre tests sans isolation explicite
- Utiliser `@BeforeEach` pour réinitialiser l'état

Exemple

```
class CompteBancaireTest {  
    private CompteBancaire compte;  
  
    @BeforeEach  
    void setUp() {  
        compte = new CompteBancaire(); // état frais pour chaque test  
    }  
}
```

| 06 · Gestion des Exceptions

Tester les cas d'erreur

Pourquoi tester les exceptions ?

- Le code de production **doit** gérer les cas d'erreur
- Les exceptions font partie du **comportement observable**
- Un test qui vérifie une exception garantit la **robustesse**
- Évite les comportements silencieux (retourner null, ignorer l'erreur)

Tester les exceptions avec JUnit 5

Méthode recommandée : `assertThrows`

```
@Test
void retirer_SoldeInsuffisant_LeveException() {
    // Arrange
    CompteBancaire compte = new CompteBancaire(100.0);

    // Act + Assert
    assertThrows(
        SoldeInsuffisantException.class,
        () -> compte.retirer(200.0)
    );
}
```

Vérifier le message de l'exception

```
@Test
void retirer_SoldeInsuffisant_MessageErreurCorrect() {
    CompteBancaire compte = new CompteBancaire(100.0);

    SoldeInsuffisantException exception = assertThrows(
        SoldeInsuffisantException.class,
        () -> compte.retirer(200.0)
    );

    assertEquals(
        "Solde insuffisant : 100.0 disponible, 200.0 demandé",
        exception.getMessage()
    );
}
```

L'ancienne approche (JUnit 4 / à éviter)

```
// ✗ Approche JUnit 4 – ne permet pas de vérifier le message
@Test(expected = SoldeInsuffisantException.class)
void retirer_SoldeInsuffisant_LeveException() {
    compte.retirer(200.0); // toute exception de ce type = test vert
}

// ✗ try-catch manuel – verbeux et fragile
@Test
void retirer_SoldeInsuffisant_LeveException() {
    try {
        compte.retirer(200.0);
        fail("Exception attendue");
    } catch (SoldeInsuffisantException e) {
        // OK
    }
}
```

Tester les cas limites

Cas à toujours envisager

- **Valeurs nulles** : paramètre `null` passé à une méthode
- **Valeurs négatives** : montant négatif, index négatif
- **Valeurs limites** : 0, Integer.MAX_VALUE, liste vide
- **Chaînes vides** : `""` vs `null` vs `" "`
- **Collections vides** : liste vide, map vide

Exemple — tests des cas limites

```
@Test
void deposer_MontantNull_LeveNullPointerException() {
    assertThrows(NullPointerException.class,
        () -> compte.deposer(null));
}

@Test
void deposer_MontantNegatif_LeveIllegalArgumentException() {
    assertThrows(IllegalArgumentException.class,
        () -> compte.deposer(-50.0));
}

@Test
void deposer_MontantZero_SoldeInchange() {
    compte.deposer(0.0);
    assertEquals(soldeInitial, compte.getSolde(), 0.001);
}
```

| 07 · Refactoring et Conception Émergente

Améliorer sans régression

Qu'est-ce que le Refactoring ?

"Refactoring est le processus de modification d'un logiciel de manière à améliorer sa structure interne sans modifier son comportement externe." — Martin Fowler

- Améliorer la **lisibilité** du code
- Réduire la **complexité**
- Éliminer la **duplication** (DRY)
- Améliorer la **modularité** et le **découplage**

Refactoring sécurisé avec TDD

Sans tests

- Refactorer = risque de régression
- Impossible de valider que le comportement est préservé
- Frein naturel à l'amélioration du code

Avec TDD

- Les tests sont le **filet de sécurité**
- Refactorer en **confiance totale**
- Les tests **échouent** si le comportement change involontairement

Les Code Smells à éliminer

Problèmes de structure

- **Long Method** : méthode de plus de 10-15 lignes
- **Duplicate Code** : même logique répétée en plusieurs endroits
- **Large Class** : classe qui fait trop de choses
- **Long Parameter List** : trop de paramètres dans une méthode

Comment les détecter ?

- Les tests révèlent souvent les code smells (setup complexe = mauvaise conception)



Refactorings courants

- **Extract Method** : extraire du code en méthode
- **Rename** : renommer pour plus de clarté
- **Extract Class** : décomposer une grande classe
- **Inline Method** : supprimer une méthode inutile
- **Move Method** : déplacer vers la bonne classe
- **Replace Magic Number** : nommer les constantes
- **Introduce Parameter Object** : regrouper des paramètres
- **Pull Up / Push Down** : reorganiser la hiérarchie

Exemple de refactoring

Avant refactoring

```
public double calculer(double m, double t) {  
    if (t < 0 || t > 1) throw new RuntimeException("err");  
    return m * t;  
}
```

Après refactoring

```
private static final double TVA_MIN = 0.0;  
private static final double TVA_MAX = 1.0;  
  
public double calculerTVA(double montantHT, double tauxTVA) {  
    validerTauxTVA(tauxTVA);  
    return montantHT * tauxTVA;  
}  
  
private void validerTauxTVA(double taux) {  
    if (taux < TVA_MIN || taux > TVA_MAX)  
        throw new IllegalArgumentException("Taux TVA invalide : " + taux);  
}
```

Conception Émergente

Principe

- Ne pas sur-concevoir **avant** d'écrire le code
- Laisser la **conception émerger** des besoins exprimés par les tests
- Principe YAGNI : *You Ain't Gonna Need It*

Processus

1. Écrire le test le plus simple possible
2. Implémenter le minimum
3. Refactorer en observant les **patterns qui émergent**
4. Répéter → la bonne architecture apparaît naturellement

Les principes SOLID sont naturellement favorisés par TDD

- **S** — Single Responsibility : une classe = une raison de changer
- **O** — Open/Closed : ouvert à l'extension, fermé à la modification
- **L** — Liskov Substitution : les sous-types respectent les contrats
- **I** — Interface Segregation : interfaces spécifiques et ciblées
- **D** — Dependency Inversion : dépendre des abstractions

TDD force à concevoir des classes à faible couplage, respectant naturellement SOLID

08 · JUnit 5 — Le Framework de Tests

L'outil incontournable

JUnit 5 — Présentation

Architecture modulaire

- **JUnit Platform** : moteur d'exécution, socle de l'écosystème
- **JUnit Jupiter** : API pour écrire les tests (annotations, assertions)
- **JUnit Vintage** : rétrocompatibilité avec JUnit 3 et 4

Pourquoi JUnit 5 ?

- Architecture extensible et moderne
- Support natif de Java 8+ (lambdas, streams)
- Annotations expressives et puissantes
- Intégration IDE et Maven/Gradle native

Configuration Maven

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.1.2</version>
</plugin>
```

Configuration Gradle

```
dependencies {  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'  
}  
  
test {  
    useJUnitPlatform()  
}
```

Annotations essentielles JUnit 5

Annotation	Rôle
<code>@Test</code>	Marque une méthode comme test
<code>@BeforeEach</code>	Exécuté avant chaque test
<code>@AfterEach</code>	Exécuté après chaque test
<code>@BeforeAll</code>	Exécuté une fois avant tous les tests
<code>@AfterAll</code>	Exécuté une fois après tous les tests
<code>@Disabled</code>	Désactive un test
<code>@DisplayName</code>	Nom lisible pour le test
<code>@Nested</code>	Tests imbriqués pour l'organisation

Structure d'une classe de test

```
@DisplayName("Tests du Compte Bancaire")
class CompteBancaireTest {

    private CompteBancaire compte;

    @BeforeEach
    void initialiserCompte() {
        compte = new CompteBancaire(500.0);
    }

    @Test
    @DisplayName("Le dépôt augmente le solde")
    void deposer_MontantPositif_AugmenteSolde() {
        compte.deposer(200.0);
        assertEquals(700.0, compte.getSolde(), 0.001);
    }

    @AfterEach
    void nettoyage() {
        // Libérer les ressources si nécessaire
    }
}
```



Les Assertions JUnit 5

```
// Égalité
assertEquals(expected, actual);
assertNotEquals(unexpected, actual);

// Booléens
assertTrue(condition);
assertFalse(condition);

// Nullité
assertNull(object);
assertNotNull(object);

// Tableaux et collections
assertArrayEquals(expectedArray, actualArray);
assertIterableEquals(expectedList, actualList);

// Exceptions
assertThrows(ExceptionType.class, () -> methode());

// Tout ou rien
assertAll(
    () -> assertEquals(1, val1),
    () -> assertEquals(2, val2)
);
```

assertAll — Grouper les assertions

```
@Test
void verifierProfilUtilisateur() {
    Utilisateur user = creerUtilisateur();

    assertAll("Profil utilisateur",
        () -> assertEquals("Alice", user.getNom()),
        () -> assertEquals("alice@email.com", user.getEmail()),
        () -> assertTrue(user.isActif()),
        () -> assertNotNull(user.getDateCreation())
    );
}
// Toutes les assertions sont évaluées, même si une échoue
```

12 Tests paramétrés — @ParameterizedTest

```
@ParameterizedTest
@ValueSource(doubles = {-1.0, -100.0, -0.001})
void deposer_MontantNegatif_LeveException(double montant) {
    assertThrows(IllegalArgumentException.class,
        () -> compte.deposer(montant));
}

@ParameterizedTest
@CsvSource({
    "100.0, 0.20, 20.0",
    "200.0, 0.10, 20.0",
    "500.0, 0.055, 27.5"
})
void calculerTVA_ParametresVaries_RetourneValeurCorrecte(
    double montant, double taux, double attendu) {
    assertEquals(attendu, calc.calculer(montant, taux), 0.001);
}
```

Sources de données paramétrées

```
// Depuis une méthode statique
@ParameterizedTest
@MethodSource("fournitMontantsValides")
void deposer_MontantsValides_PasException(double montant) {
    assertDoesNotThrow(() -> compte.deposer(montant));
}

static Stream<Double> fournitMontantsValides() {
    return Stream.of(0.01, 1.0, 100.0, 1_000_000.0);
}

// Depuis un enum
@ParameterizedTest
@EnumSource(TypeCompte.class)
void creerCompte_TousTypes_ReussitSansException(TypeCompte type) {
    assertNotNull(new CompteBancaire(type));
}
```

Tests imbriqués — @Nested

```
@DisplayName("Compte Bancaire")
class CompteBancaireTest {

    @Nested
    @DisplayName("Opérations de dépôt")
    class DepotTest {
        @Test void depot_MontantPositif_Reussit() { }
        @Test void depot_MontantNegatif_LeveException() { }
    }

    @Nested
    @DisplayName("Opérations de retrait")
    class RetraitTest {
        @Test void retrait_SoldeSuffisant_Reussit() { }
        @Test void retrait_SoldeInsuffisant_LeveException() { }
    }
}
```

Timeout et tests conditionnels

```
// Test avec timeout
@Test
@Timeout(value = 500, unit = TimeUnit.MILLISECONDS)
void rechercherUtilisateur_DoitRepondreRapidement() {
    service.rechercherParNom("Alice");
}

// Test conditionnel selon l'OS
@Test
@EnabledOnOs(OS.LINUX)
void testSpecifiqueLinux() { }

// Test conditionnel selon la version Java
@Test
@EnabledForJreRange(min = JRE.JAVA_17)
void testFonctionnaliteJava17() { }

// Test désactivé temporairement
@Test
@Disabled("En attente de fix du bug #1234")
void testDesactive() { }
```

@TestMethodOrder — Ordre d'exécution

```
// Par défaut : ordre déterministe mais non garanti
// Pour les tests d'intégration séquentiels :

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class ProcessusInscriptionTest {

    @Test @Order(1)
    void creerCompteUtilisateur() { }

    @Test @Order(2)
    void validerEmail() { }

    @Test @Order(3)
    void configurerProfil() { }
}

// Attention : l'ordre ne doit pas être nécessaire pour les tests unitaires !
```

@Tag — Catégoriser les tests

```
@Test
@Tag("rapide")
@Tag("unitaire")
void testRapide() { }

@Test
@Tag("lent")
@Tag("integration")
void testIntegration() { }
```

```
<!-- Exécuter uniquement les tests "unitaire" en CI -->
<plugin>
  <configuration>
    <groups>unitaire</groups>
    <excludedGroups>integration</excludedGroups>
  </configuration>
</plugin>
```

| 09 · TestNG et Alternatives

L'écosystème de tests Java

VS JUnit 5 vs TestNG

Fonctionnalité	JUnit 5	TestNG
Tests paramétrés	✓ Natif	✓ DataProvider
Tests en parallèle	✓ Config	✓ Natif
Groupe/Tags	✓ @Tag	✓ @Groups
Dépendances entre tests	✗	✓ @DependsOnMethods
Listeners	✓ Extension API	✓ Listeners
Popularité Java	★★★★★	★★★

TestNG — Présentation

```
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.8.0</version>
  <scope>test</scope>
</dependency>
```

Points forts de TestNG

- Tests en **parallèle** natifs et configurables
- **DataProvider** : tests paramétrés puissants
- **Grouper** : organisation fine des suites de tests
- **Dépendances** entre tests (adapté aux tests d'intégration)

Exemple TestNG

```
public class CalculeurTVATest {

    @DataProvider(name = "donneesTVA")
    public Object[][] fournirDonnees() {
        return new Object[][] {
            {100.0, 0.20, 20.0},
            {200.0, 0.10, 20.0}
        };
    }

    @Test(dataProvider = "donneesTVA")
    public void calculer_ParametresVaries_RetourneValeur(
        double montant, double taux, double attendu) {
        assertEquals(calc.calculer(montant, taux), attendu, 0.001);
    }
}
```

Outillage complémentaire

- **AssertJ** : assertions fluides et lisibles
- **Hamcrest** : matchers expressifs
- **Mockito** : mocking d'objets
- **WireMock** : simulation de services REST
- **Testcontainers** : bases de données en Docker pour les tests
- **Awaitility** : tests asynchrones
- **Faker** : génération de données de test
- **Archunit** : tests d'architecture

AssertJ — Assertions fluides

```
// JUnit 5 standard
assertEquals("Alice", user.getNom());
assertTrue(users.size() > 0);
assertNotNull(result);

// AssertJ – plus lisible
assertThat(user.getNom()).isEqualTo("Alice");
assertThat(users).isNotEmpty().hasSize(3);
assertThat(result).isNotNull().assertInstanceOf(Commande.class);
assertThat(montant).isBetween(0.0, 1000.0);
assertThat(message).startsWith("Erreur").contains("invalide");
```

Hamcrest — Matchers

```
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.*;

// Exemple d'utilisation
assertThat(compte.getSolde(), is(700.0));
assertThat(liste, hasSize(3));
assertThat(liste, contains("A", "B", "C"));
assertThat(liste, hasItem("A"));
assertThat(nom, startsWith("Al"));
assertThat(valeur, both(greaterThan(0)).and(lessThan(100)));
```

| 10 · Bonnes Pratiques JUnit

Écrire des tests maintenables

✓ Les F.I.R.S.T. en pratique

Fast — Tests rapides

```
// ✗ Test lent (accès réseau ou BDD)
@Test
void rechercherUtilisateur_AccesBDD() {
    Repository repo = new DatabaseRepository("jdbc:...");
    // lent !
}

// ✓ Test rapide (mock)
@Test
void rechercherUtilisateur_RapideAvecMock() {
    UserRepository repo = mock(UserRepository.class);
    when(repo.findById(1L)).thenReturn(new User("Alice"));
    // rapide !
}
```

✓ Organisation des tests

Structure recommandée du projet

```
src/  
├── main/java/  
│   └── com/exemple/  
│       ├── CompteBancaire.java  
│       └── service/  
│           └── VirementService.java  
└── test/java/  
    ├── com/exemple/  
    │   ├── CompteBancaireTest.java  
    │   └── service/  
    │       └── VirementServiceTest.java
```

Convention : même package, dans `src/test/` → accès aux membres `package-private`

Nommage des classes de test

Convention standard

```
// Classe testée
public class CalculateurTVA {}

// Classe de test
class CalculateurTVATest {} // ✅ Recommandé
// ou
class CalculateurTVAShould {} // ✅ Style BDD
```

Nommage des méthodes

```
// Pattern : methodeName_Contexte_ResultatAttendu
void calculer_TauxNormal_RetourneValeurCorrecte() {}
void calculer_TauxNegatif_LeveIllegalArgumentException() {}
```

❌ Anti-patterns à éviter (1/2)

Tests qui testent trop

```
// ❌ Un test qui fait plusieurs choses
@Test
void testComplet() {
    compte.deposer(100);
    assertEquals(100, compte.getSolde());
    compte.retirer(50);
    assertEquals(50, compte.getSolde()); // Si la 1ère assertion échoue ?
}
```

Tests qui dépendent de l'ordre

```
// ❌ Tests dépendants entre eux
static double soldeShared = 0;

@Test void test1_Deposer() { soldeShared = 100; }
@Test void test2_Retirer() { assertEquals(100, soldeShared); } // Fragile !
```

❌ Anti-patterns à éviter (2/2)

Tests sans assertions

```
// ❌ Test qui ne vérifie rien (toujours vert)
@Test
void testSansAssertion() {
    compte.deposer(100); // Pas d'assertEquals → test inutile
}
```

Tests trop couplés à l'implémentation

```
// ❌ Teste l'implémentation interne, pas le comportement
@Test
void testImplementationInterne() {
    verify(mockList, times(3)).add(any()); // Testé = nombre d'appels
    // Si l'implémentation change, test cassé même si comportement OK
}
```



Couverture de code

Métriques de couverture

- **Line coverage** : pourcentage de lignes exécutées
- **Branch coverage** : pourcentage de branches (if/else) testées
- **Method coverage** : pourcentage de méthodes testées
- **Class coverage** : pourcentage de classes testées

Objectif recommandé

- Viser **80-90%** de couverture de lignes
- **100% n'est pas toujours utile** (getters, code généré)
- La qualité des tests > la quantité

JaCoCo — Outil de couverture

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.10</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

TP 1 — Premiers Tests JUnit

Objectif

Concevoir et intégrer des tests dans le cycle de développement

Scénario

Implémenter une **calculatrice de factures** avec TDD :

- Calculer le montant HT, TVA et TTC
- Appliquer des remises par palier
- Gérer les cas d'erreur (montant négatif, taux invalide)

TP 1 — Instructions (1/2)

Étape 1 — Setup du projet

```
mvn archetype:generate \  
-DgroupId=com.formation.tdd \  
-DartifactId=tp-junit \  
-DarchetypeArtifactId=maven-archetype-quickstart
```

Étape 2 — Ajouter JUnit 5

Modifier le `pom.xml` pour inclure JUnit Jupiter

Étape 3 — Écrire les tests AVANT le code

Commencer par `CalculatriceFactureTest.java`

TP 1 — Instructions (2/2)

Tests à implémenter

```
// Cas nominaux
void calculerTTC_TauxNormal_RetourneValeurCorrecte()
void calculerTTC_TauxReduit_RetourneValeurCorrecte()
void appliquerRemise_PalierAtteint_AppliqueRemise()

// Cas limites
void calculerTTC_MontantZero_RetourneZero()
void calculerTTC_MontantNegatif_LeveException()
void calculerTTC_TauxInvalide_LeveException()
```

Livrable

Projet Maven avec tests verts + couverture JaCoCo > 85%

| 11 · Bonnes Pratiques Agiles

TDD et développement Agile

TDD dans un contexte Agile

Le Sprint et TDD

- Les **User Stories** deviennent des scénarios de tests
- Les **critères d'acceptation** = tests d'acceptation
- La **Definition of Done** inclut les tests automatisés
- La **vélocité** s'améliore grâce à moins de régressions

User Story → Test

User Story

"En tant qu'utilisateur, je veux pouvoir me connecter avec mon email et mot de passe, afin d'accéder à mon espace personnel."

Critères d'acceptation → Tests

```
void connexion_IdentifiantsValides_AccesAutorise()  
void connexion_MotDePasseIncorrect_AccesRefuse()  
void connexion_CompteBloqueApres3Echecs_LanceAlarme()  
void connexion_CompteInactif_AccesRefuse()
```

TDD et bases de données (SGBDR)

Problèmes des tests avec BDD

- Lents (connexion réseau)
- Dépendants de l'état de la base
- Difficiles à rendre répétables
- Risque de polluer les données de prod

Solutions

- **Repository pattern** : isoler l'accès BDD derrière une interface
- **In-memory database** : H2, HSQLDB pour les tests
- **Testcontainers** : vraie BDD en Docker, state fresh

Repository Pattern pour les tests

```
// Interface testable
public interface UserRepository {
    User findById(Long id);
    void save(User user);
}

// Implémentation de production
public class JpaUserRepository implements UserRepository { ... }

// Fausse implémentation pour les tests
public class InMemoryUserRepository implements UserRepository {
    private Map<Long, User> store = new HashMap<>();
    public User findById(Long id) { return store.get(id); }
    public void save(User u) { store.put(u.getId(), u); }
}
```

TDD et interfaces graphiques

Défis des tests d'interface

- Les UI sont difficiles à tester unitairement
- Dépendances au rendu graphique
- Tests fragiles (coordonnées pixel, textes statiques)

Stratégies

- **Patron MVP/MVVM** : séparer logique et présentation
- Tester la **logique** (Presenter/ViewModel) sans l'UI
- Tests E2E avec **Selenium** ou **Playwright** pour l'interface
- **Page Object Pattern** : encapsuler les interactions UI



MVP Pattern — testabilité

```
// Presenter testable sans UI
public class ConnexionPresenter {
    private final UserService userService;
    private ConnexionView view;

    public void onConnexion(String email, String mdp) {
        try {
            User user = userService.connector(email, mdp);
            view.afficherSucces(user.getNom());
        } catch (ConnexionException e) {
            view.afficherErreur(e.getMessage());
        }
    }
}

// Test du Presenter avec Mock de la View
@Test
void onConnexion_IdentifiantsValides_AfficheSucces() {
    when(userService.connector("alice@ex.com", "pass")).thenReturn(alice);
    presenter.onConnexion("alice@ex.com", "pass");
    verify(view).afficherSucces("Alice");
}
```

Tester les contrôleurs REST

```
// Spring Boot Test – test de contrôleur
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired MockMvc mockMvc;
    @MockBean UserService userService;

    @Test
    void getUser_IdExistant_RetourneUser() throws Exception {
        when(userService.findById(1L)).thenReturn(new User("Alice"));

        mockMvc.perform(get("/api/users/1"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.nom").value("Alice"));
    }
}
```

Intégration Continue et TDD

Pipeline CI avec tests

Commit → Build → Tests Unitaires → Tests Intégration → Rapport

Rapides
(< 1 minute)

Moyens
(< 5 min)

Couverture
JaCoCo

Règles du CI avec TDD

- **Aucun commit** sans tests verts
- **Pipeline bloquant** : build cassé = correction prioritaire
- **Feedback rapide** : notification en moins de 5 minutes



Métriques qualité à surveiller

Métrique	Objectif recommandé
Couverture de lignes	> 80%
Couverture de branches	> 70%
Temps d'exécution tests unitaires	< 1 minute
Temps d'exécution intégration	< 10 minutes
Nombre de tests échoués en CI	0 (bloquant)
Complexité cyclomatique	< 10 par méthode

TP 2 — TDD avec SGBDR

Objectif

Implémenter un service de gestion d'utilisateurs avec TDD et tests de persistance

Scénario

Créer un `UserService` qui :

- Crée un utilisateur (validation + persistance)
- Recherche par email (unicité garantie)
- Désactive un compte (soft delete)

Stack technique

JUnit 5 + H2 en mémoire + Spring Data JPA (ou JDBC simple)

TP 2 — Architecture cible

```
// Interface Repository (testable)
interface UserRepository {
    User save(User user);
    Optional<User> findByEmail(String email);
    void deactivate(Long userId);
}

// Service (logique métier) – testé avec mocks
class UserService {
    private final UserRepository repository;
    public User creer(CreateUserRequest req) { ... }
}

// Repository réel (testé avec H2)
class JdbcUserRepository implements UserRepository { ... }
```

| 12 · Synthèse du Jour 1

Ce que nous avons appris

Bilan — Jour 1 (1/2)

Fondamentaux TDD

- La **pyramide des tests** et la **valeur économique** des tests
- Le cycle **Red** → **Green** → **Refactor** et son rythme
- Le patron **AAA / Given-When-Then** pour structurer les tests
- Les **cas limites** et la gestion des **exceptions**

Conception

- **Refactoring sécurisé** grâce aux tests comme filet
- **Conception émergente** et principes SOLID
- L'impact du TDD sur la **qualité architecturale**

Bilan — Jour 1 (2/2)

JUnit 5

- Annotations essentielles : `@Test` , `@BeforeEach` , `@Nested` , `@Tag`
- Tests **paramétrés** avec `@ParameterizedTest`
- Assertions complètes : `assertEquals` , `assertThrows` , `assertAll`
- **AssertJ** pour des assertions fluides

Bonnes pratiques

- Nommage **expressif** des tests
- Tests **FIRST** : Fast, Independent, Repeatable, Self-Validating, Timely
- Anti-patterns à éviter



Programme — Jour 2

Ce qui nous attend demain

- **Objets Mock et Stub** : théorie et pratique
- **Mockito** en détail : when, verify, ArgumentCaptor
- **Techniques d'écriture** de tests avancées
- **Fixtures** et qualité du code de test
- **Styles de TDD** : classique vs London
- **Travaux pratiques** intensifs Mockito



Ressources recommandées

Livres fondamentaux

- *Test-Driven Development by Example* — Kent Beck
- *Growing Object-Oriented Software Guided by Tests* — Freeman & Pryce
- *Refactoring* — Martin Fowler
- *Clean Code* — Robert C. Martin

En ligne

- **JUnit 5 User Guide** : junit.org/junit5/docs
- **Mockito Docs** : site.mockito.org
- **AssertJ** : assertj.github.io

Exercices du soir

Pour consolider les acquis

1. Implémenter un **gestionnaire de liste de tâches** (ToDoList) en TDD pur
2. Pratiquer les **3 phases** du cycle sur chaque fonctionnalité
3. Vérifier la **couverture** avec JaCoCo
4. Identifier **3 code smells** dans votre code et les refactorer

Objectif : 20-30 minutes de pratique autonome



Fin du Jour 1

Excellente première journée !

Rendez-vous demain pour les Mocks, Stubs et Mockito



QCM — Vérification des acquis

Question 1

Dans le cycle TDD, quelle est la bonne séquence ?

- A) Green → Red → Refactor
- B) Red → Green → Refactor ☒
- C) Refactor → Red → Green
- D) Green → Refactor → Red

Question 2

Que signifie le pattern AAA dans un test ?

- A) Assert, Apply, Arrange
- B) **Arrange, Act, Assert** ✓
- C) Analyze, Apply, Assess
- D) Arrange, Assert, Act

Question 3

Quelle annotation JUnit 5 permet de grouper plusieurs assertions ?

- A) `@TestGroup`
- B) `@AssertGroup`
- C) `assertAll()` ✓
- D) `@MultiAssert`

Question 4

Que signifie FIRST pour les tests ?

- A) Focused, Independent, Reliable, Stable, Tested
- B) **Fast, Independent, Repeatable, Self-Validating, Timely** ✓
- C) Fast, Isolated, Robust, Simple, Tested
- D) Focused, Isolated, Reliable, Stable, Timely

Question 5

Quelle est la bonne pratique pour tester une méthode qui lève une exception en JUnit 5 ?

- A) `@Test(expected = ...)`
- B) try-catch manuel
- C) `assertThrows()` ✓
- D) `@ExpectedException`



Pause & Questions

N'hésitez pas à poser vos questions !

Formation TDD Java · Jour 1 · © 2026